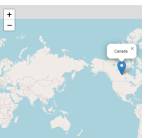
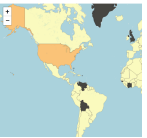
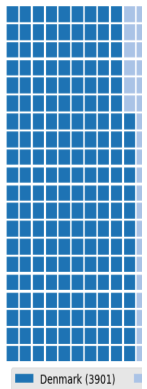
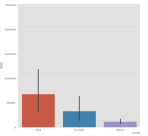
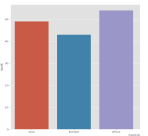
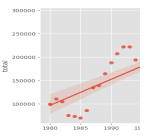




Data Visualization with Python

Cheat Sheet : Maps, Waffles, WordCloud and Seaborn

Function	Description	Syntax	Example	Visual
Folium				
Map	Create a map object with specified center coordinates and zoom level.	<code>folium.Map(location=[lat, lon], zoom_start=n)</code>	<code>world_map = folium.Map() canada = folium.Map(location=[56.130, -106.35], zoom_start=4)</code>	
Marker	Add a marker to the map with custom icon, popup, and tiles Tiles as Stamen Toner	<code>folium.Marker(location=[lat, lon], popup='Marker Popup', tiles='Stamen Toner').add_to(map)</code>	<code>folium.Marker(location=[556.130, -106.35], tooltip='Marker', tiles='Stamen Toner').add_to(world_map)</code>	
	Tiles as Stamen Terrain	<code>folium.Marker(location=[lat, lon], popup='Marker Popup', tiles='Stamen Terrain').add_to(map)</code>	<code>folium.Marker(location=[556.130, -106.35], tooltip='Marker', tiles='Stamen Terrain').add_to(world_map)</code>	
Circle	Add a circle to the map with specified radius, color, and fill opacity.	<code>folium.features.CircleMarker(location=[lat, lon], radius=n, color='red', fill_opacity=n).add_to(map)</code>	<code>folium.features.CircleMarker(location=[56.130, -106.35], radius=1000, color='red', fill_opacity=0.5).add_to(world_map)</code>	
Choropleth	Create a choropleth map based on a GeoJSON file and a specified data column.	<code>folium.Choropleth(geo_data='path/to/geojson_file', data=df, columns=['region', 'value_column'], key_on='feature.properties.id', fill_color='YlGnBu', fill_opacity=0.7, line_opacity=0.2, legend_name='Legend').add_to(map)</code>	<code>world_map.choropleth(geo_data=world_geo, data=df_can, columns=['Country', 'Total'], key_on='feature.properties.name', fill_color='YlOrRd', fill_opacity=0.7, line_opacity=0.2, legend_name='Immigration to Canada')</code>	
PyWaffle				
Waffle	Create a waffle chart based on values and categories.	<code>plt.figure(FigureClass = Waffle, rows = 20, columns = 30, values = values) waffle_chart = waffle.Waffle(values=[value1, value2, ...], rows=n, columns=n)</code>	<code>plt.figure(FigureClass = Waffle, rows = 20, columns = 30, values = df_dsn['Total'], cmap_name = 'tab20', legend = {'labels': label, 'loc': 'lower left', 'bbox_to_anchor': (0, -0.1), 'ncol': 3})</code>	
Legend	Add a legend to the waffle chart.	<code>waffle_chart.legend(loc='upper left', bbox_to_anchor=(1, 1))</code>		
Title	Add a title to the waffle chart.	<code>waffle_chart.set_title('Waffle Chart Title')</code>		

Function	Description	Syntax	Example	Visual
Labels	Add labels to the waffle chart.	<code>waffle_chart.set_labels(['Label 1', 'Label 2', ...])</code>		
WordCloud				
WordCloud	Create a word cloud object based on text data.	<code>wordcloud = WordCloud().generate(text_data)</code>	<pre>alice_wc = WordCloud(background_color='white', max_words=2000, mask=alice_mask, stopwords=stopwords) alice_wc.generate(alice_novel) plt.imshow(alice_wc, interpolation='bilinear')</pre>	
Generate	Generate the word cloud based on the text data.	<code>wordcloud.generate(text_data)</code>		
Display	Display the word cloud using matplotlib or other plotting libraries.	<code>plt.imshow(wordcloud, interpolation='bilinear')</code>		
Options	Set various options for the word cloud, such as font, colors, mask, and stopwords.	<code>wordcloud = WordCloud(font_path='path/to/font_file', background_color='white', colormap='Blues', mask=mask_image, stopwords=stopwords).generate(text_data)</code>		
Seaborn				
barplot	Create a bar plot to visualize the relationship between a categorical variable and a numeric variable.	<code>sns.barplot(x='x_variable', y='y_variable', data=dataframe)</code>	<code>sns.barplot(x='Continent', y='Total', data=df_can1)</code>	
countplot	Create a count plot to display the frequency of each category in a categorical variable.	<code>sns.countplot(x='category', data=dataframe)</code>	<code>sns.countplot(x='Continent', data=df_can)</code>	
regplot	Create a scatter plot with a linear regression line to visualize the relationship between two numeric variables.	<code>sns.regplot(x='x_variable', y='y_variable', data=dataframe)</code>	<code>sns.regplot(x='year', y='total', data=df_tot)</code>	

Author(s)

Changelog

Date	Version	Changed by	Change Description
2023-06-18	0.1	Dr. Pooja	Initial version created